CS388 Midterm, Fall 2022 — Worked Solutions

Unofficial solutions with explanations · prepared as a study aid · October 2026

> **NOTE**
> These are my worked solutions, not the official answer key. Assumptions are stated where a question is ambiguous.

Part 1 — Multiple choice / short answer (30 pts)

(1) All skip-gram (word, context) pairs for "the cat ran away", window ±2.

C — every word paired with each word within distance 2, in both directions: the→{cat, ran}; cat→{the, ran, away}; ran→{the, cat, away}; away→{cat, ran}. That's 10 pairs. Contexts are single words, not phrases (which rules out E).

(2) HMM tag transition probabilities are…

D — P(tag | preceding tag), not conditioned on words (first-order Markov). A is emission. C would be a higher-order model.

(3) The plotted S-shaped curve goes from 0 (left) to 1 (right).

B — sigmoid: → 0 as x → −∞ and → 1 as x → +∞. (tanh would go from −1 to 1.)

(4) Which preprocessing steps reduce a unigram BoW feature space?

I, III, IV, V — lowercasing merges case variants; stemming merges inflections; stopword removal deletes features; tokenizing punctuation merges "great!" / "great." / "great" into one type. II (bigrams) enlarges the space. VI (tf-idf) changes values, not dimensionality.

(5) The gender bias Bolukbasi et al. remove?

A — correlation of gender-neutral words with the he–she axis (gender direction) in embedding space. Legitimately gendered associations like king/queen (D) are meant to be kept.

(6) Best FFNN implementation for sentiment?

B — 50-dim embedding → [50×75] Linear → Tanh → [75×2] Linear → log-softmax. Dimensions chain, there's a nonlinearity between the linear layers, and the output is 2 classes.
A has a width-2 hidden layer with ReLU (a needless bottleneck, and ReLU before the output layer can zero things). C has a dimension mismatch (300 vs 50). D has no nonlinearity, so it collapses to linear. E applies Tanh before the softmax, squashing logits into [−1, 1].

(7) In forward-backward, what on its own prevents double-counting emissions?

III — count the emission at the current step in the forward recurrence and start the backward recurrence's emissions at the next step. Then $\alpha_i(s)\beta_i(s)$ includes each emission exactly once. Summing vs maxing, exponentiating, normalizing, or using log space doesn't address this.

(8) P(J/big N/car).

1/36 ≈ 0.0278
$P(J)\,P(\text{big}|J)\,P(N|J)\,P(\text{car}|N)\,P(\text{STOP}|N) = \frac{2}{3}\cdot\frac{1}{2}\cdot\frac{1}{4}\cdot\frac{2}{3}\cdot\frac{1}{2} = \frac{1}{36}.$

(9) Number of possible (nonzero) tag sequences for "big convertible car".

2 — big: only J (P(big|N) = 0); car: only N; convertible: J or N. Check transitions: J→J→N ✓ and J→N→N ✓ (both nonzero). (Probabilities: JJN = 1/288, JNN = 1/216.)

(10) "big car convertible".

1 — big = J, car = N. After N, P(J|N) = 0, so convertible must be N: only J N N.

(11) Most similar pair by dot product: good=[0,0,3], bad=[0,1,3], the=[10,10,10].

C (bad, the) — good·bad = 9, good·the = 30, bad·the = 40.

(12) By cosine similarity.

A (good, bad) — cos(good,bad) = 9/(3·√10) = 0.949; cos(good,the) = 30/(3·17.32) = 0.577; cos(bad,the) = 40/(3.162·17.32) = 0.730.

(13) Dot product after normalizing all vectors to unit length.

A (good, bad) — the dot product of unit vectors equals cosine similarity.

Part 2 — Short answer (16 pts)

2a. Skip-gram vs CBOW.

Skip-gram predicts each context word from the center word, $P(c \mid w)$. CBOW predicts the center word from the sum or average of its context word vectors, $P(w \mid c_{-k..k})$.

2b. One advantage and one disadvantage of structured prediction (CRF) vs per-token classification.

Advantage: it models dependencies between labels, so outputs are coherent (e.g., it never predicts I-PER after O) and decoding is joint over the sequence.
Disadvantage: training and inference are slower and more complex (Viterbi / forward-backward cost $O(nk^2)$ and need dynamic programming), and the model is harder to implement.

2c. A new, very negative slang word appears. Two ways to update a unigram BoW model.

(1) Add the new examples (labeled negative) to the training data, extend the vocabulary with the new word, and retrain or continue training (fine-tune) so it gets a negative weight. (2) Manually set the weight for the new feature to a strongly negative value, or map it to a known similar word or lexicon feature (e.g., a "negative-lexicon word" feature). Other acceptable answers: use subword or pretrained embedding features that generalize to the new word.

2d. A dialect where the words of each sentence are alphabetized. Dependency or constituency?

Dependency parsing. Alphabetizing destroys word order, so constituents are no longer contiguous spans, which constituency trees require. Dependency arcs relate head and dependent words regardless of position (non-projective parsing handles crossing arcs), so the structure can still be represented.

Part 3 — Long answer

Q3 (12 pts) Points (1,0,+), (1,1,+), (0,1,−); classify positive iff $w^\top x > 0$; no bias.

(a) One pass from w = (0,0):

x	score	pred	gold	w after
(1,0)	0	− (0 is not > 0)	+	(1,0)
(1,1)	1	+	+	(1,0)
(0,1)	0	−	−	(1,0)

w = (1, 0)

(b) A weight vector separates the original data iff $w_1 > 0$, $w_1 + w_2 > 0$, and $w_2 \le 0$. Write $w = w_1(1, -\alpha)$ with $w_1 > 0$, $\alpha \in [0, 1)$. A new negative point $p = (x_1, x_2)$ needs $w \cdot p \le 0$ for some such w, i.e. $x_1 - \alpha x_2 \le 0$ for some $\alpha \in [0, 1)$. This is impossible exactly when $x_1 > 0$ and $x_1 \ge x_2$.
Perceptron will not converge if the 4th (negative) point lies in {$x_1 > 0$ and $x_2 \le x_1$} — the region right of the x_2-axis on or below the line $x_2 = x_1$. It includes the positive cone between (1,0) and (1,1), and everything below it. (Points there are forced to score > 0 by every separator of the original data.)

(c) (4,2) satisfies $x_1 > 0$ and $x_2 \le x_1$, so it isn't linearly separable. **Yes, a neural network exists.** All points are distinct, and a sufficiently wide (or deep) network with a nonlinearity can carve out a nonlinear boundary (e.g., one that labels (4,2) negative while (1,0) and (1,1) stay positive). A finite set of distinct, consistently labeled points can always be fit.

Q4 (17 pts) PCFG: S→PRP VP, VP→V NP, NP→J N (all 1.0); V→like/eat (.5 each); J→green/red/verde/roja (.25 each); PRP→I, N→salsa (1.0).

(a) (S (PRP I) (VP (V like) (NP (J green) (N salsa))))
(b) 1.0 (S) × 1.0 (PRP→I) × 1.0 (VP) × 0.5 (V→like) × 1.0 (NP) × 0.25 (J→green) × 1.0 (N→salsa) = **0.125**
(c) NP → N J (allows "salsa verde").
(d) e.g. **"I like salsa green"** (or "I eat salsa red"). English adjectives can now appear post-nominally.
(e) Split by language:

```
S     → PRP VP          VP → V NP
NP    → NP-EN | NP-ES
NP-EN → J-EN N          NP-ES → N J-ES
J-EN  → green | red      J-ES  → verde | roja
N → salsa   V → like | eat   PRP → I
```

Now English adjectives only precede salsa and Spanish ones only follow it.

Q5 (16 pts) Skip-gram over bigrams (words = bigrams b, contexts = unigrams).

(a) $P(\text{context} = y \mid \text{word} = b) = \dfrac{\exp(v_b^\top c_y)}{\sum_{y' \in V} \exp(v_b^\top c_{y'})}$
(b) O(|V| d) — the normalizer needs |V| dot products of length d.

(c) **O(M N E)** — M examples × N epochs × E per step.
(d) **|B|·d + |V|·d** — one d-vector per attested bigram (word side) and one per unigram (context side).
(e) $P(y \mid b = (w_1, w_2)) = \dfrac{\exp((v_{w_1} + v_{w_2})^\top c_y)}{\sum_{y' \in V} \exp((v_{w_1} + v_{w_2})^\top c_{y'})}$, with unigram word vectors v_w.
(f) **O(|V| d)** — the extra vector addition is O(d), dominated by the normalizer.
(g) **2|V|·d** — |V| word vectors + |V| context vectors (no longer depends on |B|).

Q6 (9 pts) NER with BIO on "It has been 53 years since NASA astronauts Neil Armstrong and Buzz Aldrin landed on the Moon".

(a) Gold tags:

```
It O  has O  been O  53 O  years O  since O  NASA B-ORG  astronauts O
Neil B-PER  Armstrong I-PER  and O  Buzz B-PER  Aldrin I-PER
landed O  on O  the O  Moon B-LOC
```

(b) Gold spans (4): NASA/ORG, Neil Armstrong/PER, Buzz Aldrin/PER, Moon/LOC.
Predicted spans (3): Neil Armstrong/PER ✓, Buzz/PER × (wrong boundary), Moon/ORG × (wrong type).
Precision = 1/3 ≈ 0.33, Recall = 1/4 = 0.25 (F1 ≈ 0.29).
(c) Ways to increase recall: (1) richer features — capitalization/word shape, character n-grams/suffixes, gazetteers (lists of names, orgs, places), or pretrained embeddings/BERT representations; (2) more or augmented training data, or a re-weighted loss / lowered bias toward O so the model predicts entity tags more readily.